

Collection

+ Generic Collections :-

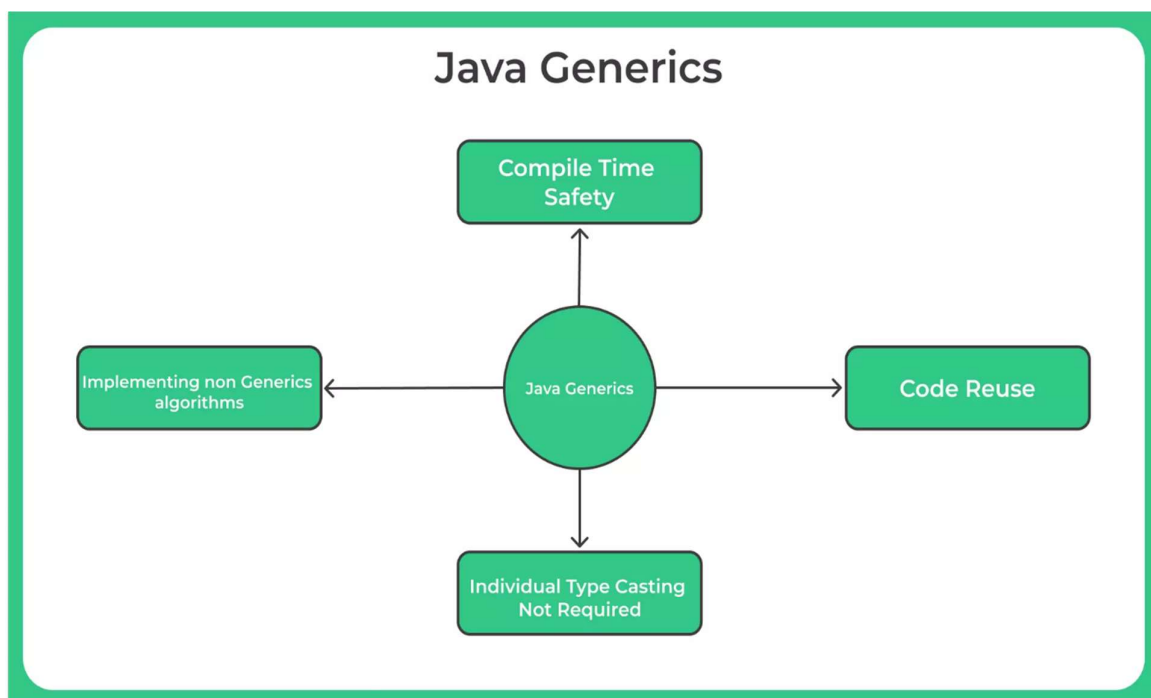
- A Generic Collection in Java allows you to specify the **type of objects** that a collection can hold.
- There are store in **same type of data**.
- It can using **Homogeneous** Object.

+ Syntax of Generic Collection :-

```
CollectionType<DataType> collectionName = new  
CollectionType<>();
```

OR

```
ArrayList<String> name = new ArrayList<String>();
```



Implementation of Generic Collection :-

Input :-

```
1 package Collection_Framework;
2
3 import java.util.ArrayList;
4 import java.util.Iterator;
5
6 public class Generic_Collection {
7
8     public static void main(String[] args) throws InterruptedException {
9
10         ArrayList<String> name = new ArrayList<String>();
11         name.add("Akshay");
12         name.add("Nikhil");
13         name.add("Akash");
14         System.out.println("Generic Collection is : "+name);
15
16         // Sorting
17         name.sort(null);
18         System.out.println("\nGeneric Collection after sorting is :"+name);
19
20         // Iteration
21         Iterator it = name.iterator();
22         System.out.println("\nSorting of Collection is : ");
23         while(it.hasNext())
24         {
25             System.out.println(it.next());
26             Thread.sleep(1000);
27         }
28     }
29 }
30 }
```

Output :-

Generic Collection is : [Akshay, Nikhil, Akash]

Generic Collection after sorting is :[Akash, Akshay, Nikhil]

Sorting of Collection is :

Akash
Akshay
Nikhil

🚦 Linked List :-

- LinkedList in Java is a class in the **java.util** package that implements both the **List** and **Deque** interfaces.
- It is a **doubly linked list**, meaning each element (node) has links to both the previous and next nodes.

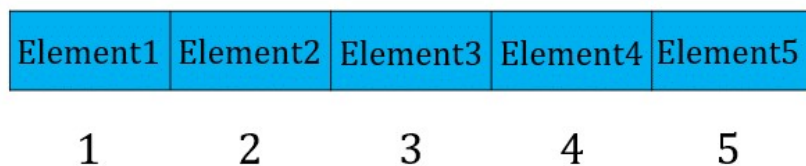
🚦 Syntax of LinkedList :-

```
LinkedList<Type> list = new LinkedList<Type>();
```

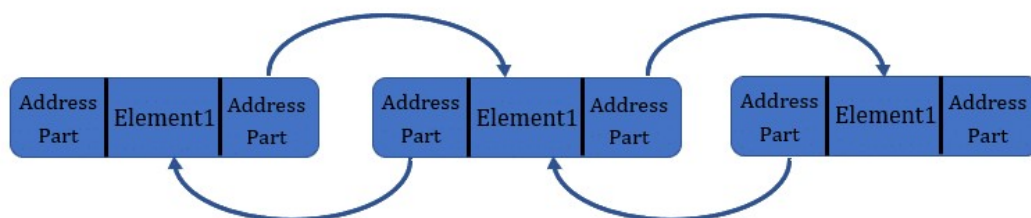
- LinkedList is ideal when your program involves **frequent insertion and deletion** of elements.
- It **maintains order**, allows **duplicates**, and provides **queue/deque functionalities**.

🚦 Difference between ArrayList and LinkedList :-

ArrayList



LinkedList



Implementation of LinkedList :-

Input :-

```
1 package Collection_Framework;
2
3 import java.util.Iterator;
4 import java.util.LinkedList;
5
6 public class linkedlist {
7
8     public static void main(String[] args) {
9
10         LinkedList l = new LinkedList();
11         l.add("nikhil@77gmail.com");
12         l.add(567);
13         l.add("Nikhil");
14         l.add("#");
15
16         System.out.println(l.offer(76));
17         l.add(81.2);
18         System.out.println("Peek : "+l.peek());
19         System.out.println("Poll : "+l.poll());
20         System.out.println("LinkedList : "+l);
21     }
22 }
```

Output :-

```
true
Peek : nikhil@77gmail.com
LinkedList : [nikhil@77gmail.com, 567, Nikhil, #, 76, 81.2]
```

✚ Vector :-

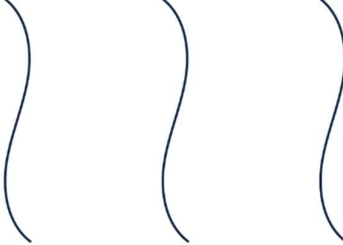
- Vector is a **legacy class** in Java that implements a **dynamic array** similar to ArrayList, but **synchronized**.
- It is part of the **java.util** package and implements the **List** interface.

✚ Syntax of Vector :-

```
Vector<Type> vector = new Vector<Type>();
```

- Vector is a **thread-safe, dynamic array**.
- It's rarely used today — replaced by **ArrayList** and **Collections.synchronizedList()** for modern Java code.
- Still useful when **synchronization** is required by default.

✚ Difference between ArrayList and Vector :-

ArrayList	Vector
 Thread1 Thread2 Thread3	 Thread1 Waiting: Thread2, Thread3

Implementation of Vector :-

Input :-

```
1 package Collection_Framework;
2
3 import java.util.Iterator;
4 import java.util.Vector;
5
6 public class Vector_demo {
7
8     public static void main(String[] args) {
9
10         Vector<Integer> rollno = new Vector<Integer>();
11         rollno.add(90);
12         rollno.add(56);
13         rollno.add(80);
14
15         // Iterator the object of collection change the value of index 1 to 60
16
17         rollno.set(1, 60);
18
19         for(Integer i:rollno)
20         {
21             System.out.println(i);
22         }
23     }
24 }
```

Output :-

90

60

80

Task 1

Input :-

```
1 package Collection_Framework;
2
3 import java.util.ArrayList;
4
5 public class task_1 {
6
7     public static void main(String[] args) {
8
9         ArrayList al = new ArrayList();
10        al.add(45);
11        al.add("Nikhil");
12        int flag=0;
13
14        for(int i=0;i<al.size();i++)
15        {
16            if(al.get(i).equals("Nikhil"))
17            {
18                flag++;
19            }
20        }
21        if(flag==0)
22        {
23            System.out.println("Nikhil object does not exist....!");
24        }
25        else
26        {
27            System.out.println("Nikhil object exist....!");
28        }
29    }
30 }
```

Output :-

Nikhil object exist....!

Task 2

Input :-

```
1 package Collection_Framework;
2
3 import java.util.ArrayList;
4
5 public class task_2 {
6
7     public static void main(String[] args) {
8
9         ArrayList al = new ArrayList();
10        al.add(45);
11        al.add("y");
12        al.add("Pune");
13        al.add(45.6);
14        al.add(987472904746353831);
15
16        for(int i=0; i<al.size();i++)
17        {
18            if(i==3)
19            {
20                al.add(i, 78923.23);
21            }
22        }
23        System.out.println("ArrayList is : "+al);
24    }
25 }
```

Output :-

```
ArrayList is : [45, y, Pune, 78923.23, 45.6, 987472904746353831]
```